Same Day Solution — Payout API Integration Guide

Base URL: `https://api.samedaysolution.in`
Version: 2.0 · October 2026

Overview

The Payout API lets you initiate bank account transfers from your application. Same Day Solution handles routing, wallet debits, charges, and settlement on your behalf.

- No third-party credentials required
- eKYC Hub penny-drop account verification via `POST /verify` (₹4.72)
- Account registration via `POST /accounts` — `register` (₹4.72) or `verify` (₹4.72 + eKYC Hub)
- Transfer requires a pre-registered account
- Transfer charges per your assigned scheme (no fee at transfer time)
- Auto-refund on failed transfers
- Webhook callbacks for every status change

Charges Summary

Charge	When	Amount
Verify (penny-drop)	`POST /verify` — eKYC Hub penny-less check; returns bank-verified name	**₹4 + 18% GST = ₹4.72**
Register	`POST /accounts` with `registration_type: "register"`	**₹4 + 18% GST = ₹4.72**
Verified Register	`POST /accounts` with `registration_type: "verify"` — eKYC Hub + register	**₹4 + 18% GST = ₹4.72**
Transfer charge	Every transfer	Per your scheme (e.g. ₹5–₹20 flat or %)

> ***Registration fee:*** *Charged once per unique account (account number + IFSC). Already-registered or already-verified accounts incur no charge.*

Before You Start (Checklist)

Contact **Same Day Solution support** to complete these one-time steps:

#	Step	Who does it
1	Generate API Key & Secret	You — Partner Dashboard → Settings → API Management
2	Whitelist your server IP	You — Partner Dashboard → API Management → Manage IP Whitelist
3	Enable Settlement / Payout	Admin enables toggle on your account
4	Get wallet balance credited	Admin credits your partner wallet
5	Scheme assignment	Admin assigns a payout charge scheme to your account

> ⚠️ *Calls from non-whitelisted IPs return **401 Unauthorized** even with a valid API key.*
> ⚠️ *Partners with **no scheme assigned** cannot initiate any transfers or account registrations.*

Authentication

Every request requires three headers:

Header	Value
`x-api-key`	Your API key (public)
`x-signature`	HMAC-SHA256 signature (see below)
`x-timestamp`	Current Unix timestamp in **milliseconds**

Signature Algorithm

```
signature = HMAC-SHA256( api_secret, payload )

For POST requests:  payload = JSON.stringify(requestBody) + timestamp
For GET requests:   payload = "" + timestamp
```

> ***Critical:*** *`JSON.stringify()` must produce **compact JSON** (no spaces, no newlines). Never sign formatted/indented JSON — it will fail.*
> *The timestamp must be within **5 minutes** of server time.*

Code Examples

Node.js

```
const crypto = require('crypto');
const axios  = require('axios');

const API_KEY    = process.env.API_KEY;
const API_SECRET = process.env.API_SECRET;
const BASE_URL   = 'https://api.samedaysolution.in';

function buildHeaders(body = null) {
  const timestamp = Date.now().toString();
  const payload   = body ? JSON.stringify(body) + timestamp : '' + timestamp;
  const signature = crypto
    .createHmac('sha256', API_SECRET)
    .update(payload)
    .digest('hex');
  return {
    'x-api-key':   API_KEY,
    'x-signature': signature,
    'x-timestamp': timestamp,
    'Content-Type': 'application/json',
  };
}
```

```
// Register account
async function registerAccount(body) {
  const { data } = await axios.post(`${BASE_URL}/api/partner/payout/accounts`, body, {
    headers: buildHeaders(body),
  });
  return data;
}

// Send payout
async function sendPayout(body) {
  const { data } = await axios.post(`${BASE_URL}/api/partner/payout/transfer`, body, {
    headers: buildHeaders(body),
  });
  return data;
}

// Check status
async function checkStatus(transactionId) {
  const { data } = await axios.get(
    `${BASE_URL}/api/partner/payout/status?transactionId=${transactionId}`,
    { headers: buildHeaders() }
  );
  return data;
}
```

PHP

```
define('API_KEY',    getenv('API_KEY'));
define('API_SECRET', getenv('API_SECRET'));
define('BASE_URL',   'https://api.samedaysolution.in');

function buildHeaders(array $body = null): array {
    $timestamp = (int)(microtime(true) * 1000);
    $payload   = $body ? json_encode($body, JSON_UNESCAPED_UNICODE) . $timestamp : '' .
$timestamp;
    $signature = hash_hmac('sha256', $payload, API_SECRET);
    return [
        'x-api-key: '    . API_KEY,
        'x-signature: ' . $signature,
        'x-timestamp: ' . $timestamp,
        'Content-Type: application/json',
    ];
}

function apiPost(string $path, array $body): array {
    $ch = curl_init(BASE_URL . $path);
    curl_setopt($ch, CURLOPT_POST, true);
    curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($body, JSON_UNESCAPED_UNICODE));
    curl_setopt($ch, CURLOPT_HTTPHEADER, buildHeaders($body));
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
```

```
    $res = curl_exec($ch);
    curl_close($ch);
    return json_decode($res, true);
}
```

Python

```
import hmac, hashlib, time, json, requests, os

API_KEY    = os.environ['API_KEY']
API_SECRET = os.environ['API_SECRET']
BASE_URL   = 'https://api.samedaysolution.in'

def build_headers(body: dict = None) -> dict:
    timestamp = str(int(time.time() * 1000))
    payload   = (json.dumps(body, separators=(',', ':')) + timestamp) if body else ('' +
timestamp)
    signature = hmac.new(API_SECRET.encode(), payload.encode(), hashlib.sha256).hexdigest()
    return {
        'x-api-key':    API_KEY,
        'x-signature':  signature,
        'x-timestamp':  timestamp,
        'Content-Type': 'application/json',
    }

def register_account(body: dict):
    return requests.post(f'{BASE_URL}/api/partner/payout/accounts',
                         json=body, headers=build_headers(body)).json()

def send_payout(body: dict):
    return requests.post(f'{BASE_URL}/api/partner/payout/transfer',
                         json=body, headers=build_headers(body)).json()

def check_status(transaction_id: str):
    return requests.get(f'{BASE_URL}/api/partner/payout/status?transactionId=
{transaction_id}',
                        headers=build_headers()).json()
```

API Reference

1. Get Wallet Balance

```
GET /api/partner/payout/balance
```

Check available balance before registering accounts or initiating transfers.

Response

```
{
  "success": true,
  "balance": 9850.00,
  "is_frozen": false,
  "freeze_reason": null
}
```

Field	Description
`balance`	Usable balance in ₹
`is_frozen`	`true` = wallet locked, contact admin

---

2. Get Supported Banks

```
GET /api/partner/payout/banks
```

Retrieve the list of supported banks for payout transfers.

Query Parameters (all optional)

Parameter	Description
`imps`	Set `true` to filter only IMPS-enabled banks
`neft`	Set `true` to filter only NEFT-enabled banks
`popular`	Set `true` to show only popular/common banks
`search`	Search by bank name (e.g. `AXIS`)

Response

```
{
  "success": true,
  "banks": [
    { "id": 1, "name": "AXIS BANK", "ifsc_prefix": "UTIB", "imps": true, "neft": true },
    { "id": 2, "name": "STATE BANK OF INDIA", "ifsc_prefix": "SBIN", "imps": true, "neft":
true }
  ],
  "total": 150,
  "imps_enabled": 142,
  "neft_enabled": 148
}
```

Field	Description
`banks`	Array of bank objects
`total`	Total number of banks returned

`imps_enabled`	Count of IMPS-supported banks
`neft_enabled`	Count of NEFT-supported banks

---

3. Verify Bank Account (Penny Drop)

```
POST /api/partner/payout/verify
```

Validate a bank account via **eKYC Hub penny-less**. Charges **₹4 + 18% GST = ₹4.72** on success. No charge on failure.

Returns the **bank-verified account holder name** so the user can confirm account details before proceeding.

Request Body

Field	Type	Required	Description
`accountNumber`	string	✅	Bank account number (9–18 digits)
`ifscCode`	string	✅	IFSC code (e.g. `UTIB0000248`)
`bankName`	string	—	Bank name (for reference in response)

Example Request

```
{
  "accountNumber": "924020056452946",
  "ifscCode": "UTIB0000248",
  "bankName": "AXIS BANK"
}
```

Response — Valid Account (HTTP 200)

```
{
  "success": true,
  "is_valid": true,
  "account_holder_name": "RAHUL SHARMA",
  "bank_name": "AXIS BANK",
  "branch_name": null,
  "verification_charges": 4.72,
  "verification_charges_detail": "₹4 + ₹0.72 GST",
  "message": "Account verified successfully via eKYC Hub. ₹4.72 charged.",
  "reference_id": "PTNR_VFY_1729083450123"
}
```

Response — Invalid Account (HTTP 400)

```
{
  "success": false,
  "error": { "code": "VALIDATION_FAILED", "message": "Account validation failed — check
account number and IFSC code." },
  "is_valid": false,
```

```
  "verification_charges": 0
}
```

Field	Description
`is_valid`	`true` if eKYC Hub confirmed account exists
`account_holder_name`	Bank-verified name — show to user for confirmation before registering
`verification_charges`	`4.72` on success; `0` on failure (no charge on invalid account or service error)
`verification_charges_detail`	`"₹4 + ₹0.72 GST"` breakdown on success

Error — Insufficient Balance (HTTP 400)

```
{
  "success": false,
  "error": { "code": "INSUFFICIENT_BALANCE", "message": "Insufficient balance. Account verification fee is ₹4.72 (₹4 + ₹0.72 GST). Available: ₹2.00" },
  "wallet_balance": 2.00,
  "verification_fee": 4.72
}
```

> ***Tip:*** *Call Verify then Register. Display the returned* `account_holder_name` *to the user. If the name matches, proceed to* `POST /accounts` *with* `registration_type: "verify"` *— the eKYC Hub call runs again inside accounts (another ₹4.72 charge), or use* `registration_type: "register"` *to skip eKYC at registration.*

---

4. Register Bank Account

```
POST /api/partner/payout/accounts
```

Register a beneficiary bank account before making transfers. Two modes via `registration_type`:

Mode	eKYC Hub	Charge	`is_bank_verified`
`register` (default)	❌ No	**₹4.72**	`false`
`verify`	✅ eKYC Hub penny-less	**₹4.72**	`true`

Upgrade: If the account is already registered (unverified), call with `registration_type: "verify"` to run eKYC and mark it bank-verified (₹4.72 charged).

Already-verified accounts: Returns success immediately — **no charge**.

> ***Recommended flow:*** `POST /verify` *(₹4.72 — get bank-verified name) → confirm name →* `POST /accounts` *with* `registration_type: "register"` *(₹4.72, skip eKYC) or* `registration_type: "verify"` *(₹4.72, runs eKYC again for a fully verified beneficiary).*

Fires `payout.account_registered` webhook on success.

Request Body

Field	Type	Required	Description
`accountNumber`	string	✅	Bank account number (9–18 digits)
`ifscCode`	string	✅	IFSC code (e.g. `UTIB0000248`)
`accountHolderName`	string	✅ for `register` / — for `verify`	Name required for plain `register`; eKYC fills it for `verify`
`bankName`	string	✅	Bank name (e.g. `AXIS BANK`)
`mobile`	string	✅	10-digit Indian mobile number
`email`	string	—	Beneficiary email address
`registration_type`	string	—	`"register"` (default) or `"verify"`

Example — Plain Register

```
{
  "accountNumber": "924020056452946",
  "ifscCode": "UTIB0000248",
  "accountHolderName": "Rahul Sharma",
  "bankName": "AXIS BANK",
  "mobile": "9876543210",
  "registration_type": "register"
}
```

Example — eKYC-Verified Register

```
{
  "accountNumber": "924020056452946",
  "ifscCode": "UTIB0000248",
  "accountHolderName": "Rahul Sharma",
  "bankName": "AXIS BANK",
  "mobile": "9876543210",
  "registration_type": "verify"
}
```

Response — New Account (₹4.72 charged)

```
{
  "success": true,
  "registration_type": "verify",
  "message": "Account verified and registered. ₹4.72 (₹4 + ₹0.72 GST) charged.",
  "verification_charge": 4.72,
  "reference_id": "KBENE_VFY_1729083450123_AB3XY",
  "account": {
    "account_number": "**********2946",
    "ifsc_code": "UTIB0000248",
    "account_holder_name": "RAHUL SHARMA",
    "bank_name": "AXIS BANK",
```

```
    "is_registered": true,
    "is_bank_verified": true
  }
}
```

Response — Already Registered (no charge)

```
{
  "success": true,
  "already_registered": true,
  "registration_type": "register",
  "verification_charge": 0,
  "message": "Account is already registered. No charge applied. Use
registration_type=\"verify\" to bank-verify this account.",
  "account": {
    "account_number": "**********2946",
    "ifsc_code": "UTIB0000248",
    "account_holder_name": "Rahul Sharma",
    "bank_name": "AXIS BANK",
    "is_registered": true,
    "is_bank_verified": false
  }
}
```

Field	Description
`registration_type`	`"register"` or `"verify"` — reflects the path taken
`verification_charge`	₹4.72 for new registrations; `0` if already registered/verified
`reference_id`	Reference ID for this transaction
`is_bank_verified`	`true` only when eKYC Hub confirmed the account
`account_holder_name`	Bank-verified name (from eKYC) when `is_bank_verified: true`; partner-supplied otherwise

---

5. List Registered Accounts

```
GET /api/partner/payout/accounts
```

Retrieve all registered beneficiary accounts for your partner account.

Response

```
{
  "success": true,
  "count": 1,
  "accounts": [
    {
      "id": "uuid-bene-id",
```

```
      "account_number": "**********2946",
      "ifsc_code": "UTIB0000248",
      "account_holder_name": "RAHUL SHARMA",
      "bank_name": "AXIS BANK",
      "mobile": "9876543210",
      "is_registered": true,
      "is_bank_verified": true,
      "created_at": "2026-10-26T10:00:00.000Z"
    }
  ]
}
```

---

6. Transfer (Initiate Payout)

```
POST /api/partner/payout/transfer
```

Debit your wallet and transfer funds to a **pre-registered** bank account. The account must be registered via `POST /accounts` first.

> ***Account must be registered.*** *Calling Transfer without a registered account returns* `ACCOUNT_NOT_REGISTERED` *(HTTP 400). Pre-registration ensures the beneficiary name, bene-code, and bank-verified flag are all correctly resolved before the wallet is debited.*

Request Body

Field	Type	Required	Description
`accountNumber`	string	✅	Must match a registered account
`ifscCode`	string	✅	Must match the registered IFSC
`accountHolderName`	string	✅	Beneficiary full name
`bankName`	string	✅	Bank name
`mobile`	string	✅	10-digit Indian mobile number
`amount`	number	✅	Transfer amount in ₹ — min ₹100, max ₹2,00,000
`email`	string	—	Beneficiary email
`paymentMode`	string	—	`IMPS` (default) or `NEFT`
`remarks`	string	—	Transaction narration

Example Request

```
{
  "accountNumber": "924020056452946",
  "ifscCode": "UTIB0000248",
  "accountHolderName": "Rahul Sharma",
  "bankName": "AXIS BANK",
  "mobile": "9876543210",
```

```
  "amount": 5000,
  "paymentMode": "IMPS",
  "remarks": "Salary payment"
}
```

Response


```
{
  "success": true,
  "message": "Transfer successful",
  "transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "partner_trans_id": "KNT17290834501234",
  "knteer_trans_id": "KSPH01035852600003",
  "utr": "565685855656",
  "status": "SUCCESS",
  "amount": 5000,
  "charges": 15,
  "total_debited": 5015,
  "payment_mode": "IMPS",
  "account_number": "***********2946",
  "account_holder_name": "RAHUL SHARMA",
  "bank_name": "AXIS BANK",
  "is_bank_verified": true,
  "remarks": "Salary payment",
  "created_at": "2026-10-26T11:15:00.000Z",
  "created_at_ist": "26/10/2026, 4:45:00 pm",
  "completed_at": "2026-10-26T11:15:12.000Z",
  "completed_at_ist": "26/10/2026, 4:45:12 pm"
}
```


Field	Description
`transaction_id`	UUID — **save this immediately** for status checks
`partner_trans_id`	Internal reference — can also be used for status checks
`status`	`SUCCESS` / `PENDING` / `FAILED`
`utr`	Bank UTR — available when `status = SUCCESS`
`charges`	Transfer charge per your scheme
`total_debited`	Total debited = amount + charges
`is_bank_verified`	`true` if the beneficiary account was eKYC Hub verified
`account_holder_name`	Name used for transfer — bank-verified name if `is_bank_verified: true`
`created_at`	ISO-8601 timestamp in UTC
`created_at_ist`	Human-readable timestamp in IST (Asia/Kolkata)
`completed_at`	Completion timestamp (null if `PENDING`)

`completed_at_ist`	Completion timestamp in IST

Optional: Idempotency Header

Add `x-idempotency-key: <unique-string>` to prevent double-debit on network retries.

```
x-idempotency-key: order-TXN-20261026-001
```

---

7. Check Transaction Status

```
GET /api/partner/payout/status?transactionId={transaction_id}
```

Check transfer status. For `PENDING` transactions, this also fetches live status from the bank and auto-updates the record.

Query Parameters (use one)

Parameter	Description
`transactionId`	UUID from Transfer response
`partnerTransId`	`partner_trans_id` from Transfer response

Response


```
{
  "success": true,
  "transaction": {
    "transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "partner_trans_id": "KNT17290834501234",
    "knteer_trans_id": "KSPH01035852600003",
    "utr": "565685855656",
    "status": "SUCCESS",
    "amount": 5000,
    "charges": 15,
    "total_debited": 5015,
    "payment_mode": "IMPS",
    "account_number": "**********2946",
    "account_holder_name": "Rahul Sharma",
    "bank_name": "AXIS BANK",
    "ifsc_code": "UTIB0000248",
    "remarks": "Salary payment",
    "created_at": "2026-10-26T11:15:00.000Z",
    "created_at_ist": "26/10/2026, 4:45:00 pm",
    "completed_at": "2026-10-26T11:15:12.000Z",
    "completed_at_ist": "26/10/2026, 4:45:12 pm"
  }
}
```


---

8. List Transactions

```
GET /api/partner/payout/status?list=true&limit=20
```

Retrieve recent payout transactions (max 50 per call).

Parameter	Default	Description
`list`	—	Must be `true`
`limit`	`20`	Records to return (max `50`)

Webhooks

Subscribe your endpoint in the admin panel → **Partner API** → your partner → **Webhooks** → tick **Knteer Payout**.

All events are signed with `X-Sameday-Signature` (HMAC-SHA256 of the body + timestamp using your webhook secret).

Webhook Events

Event	When
`payout.account_registered`	After `POST /accounts` — new account registered, ₹4.72 charged
`payout.success`	Transfer completed, UTR available
`payout.failed`	Transfer failed, wallet auto-refunded
`payout.pending`	Transfer initiated but bank not yet confirmed
`payout.refunded`	Explicit refund credited

Example Payload — `payout.account_registered`

```
{
  "event": "payout.account_registered",
  "data": {
    "partner_trans_id": "KBENE_1729083450123_AB3XY",
    "account_number": "**********2946",
    "account_holder_name": "Rahul Sharma",
    "bank_name": "AXIS BANK",
    "ifsc_code": "UTIB0000248",
    "verification_charge": 4.72,
    "bene_registered": true,
    "timestamp": "2026-10-26T10:00:00.000Z"
  }
}
```

Example Payload — `payout.pending`

```
{
  "event": "payout.pending",
```

```
  "data": {
    "transaction_id": "4e537f55-3f68-4ad8-bb55-374c4887082e",
    "partner_trans_id": "KNT17290834501234",
    "provider_txn_id": null,
    "utr": null,
    "status": "PENDING",
    "amount": 49969,
    "charges": 18,
    "total_debited": 49987,
    "payment_mode": "IMPS",
    "account_number": "***********2946",
    "account_holder_name": "Rahul Sharma",
    "bank_name": "AXIS BANK",
    "ifsc_code": "UTIB0000248",
    "failure_reason": null,
    "timestamp": "2026-10-26T11:15:00.000Z"
  }
}
```


Example Payload — `payout.success`


```
{
  "event": "payout.success",
  "data": {
    "transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "partner_trans_id": "KNT17290834501234",
    "provider_txn_id": "KSPH01035852600003",
    "utr": "565685855656",
    "status": "SUCCESS",
    "amount": 5000,
    "charges": 15,
    "total_debited": 5015,
    "payment_mode": "IMPS",
    "account_number": "***********2946",
    "account_holder_name": "Rahul Sharma",
    "bank_name": "AXIS BANK",
    "ifsc_code": "UTIB0000248",
    "failure_reason": null,
    "timestamp": "2026-10-26T16:45:12.000Z"
  }
}
```


Example Payload — `payout.failed`


```
{
  "event": "payout.failed",
  "data": {
    "transaction_id": "c3d4e5f6-a7b8-9012-cdef-012345678902",
    "partner_trans_id": "KNT17290834509876",
```

```
    "provider_txn_id": "KSPH091025600003",
    "utr": null,
    "status": "FAILED",
    "amount": 5000,
    "charges": 15,
    "total_debited": 5015,
    "payment_mode": "IMPS",
    "account_number": "***********2946",
    "account_holder_name": "Rahul Sharma",
    "bank_name": "AXIS BANK",
    "ifsc_code": "UTIB0000248",
    "failure_reason": "Transfer failed",
    "timestamp": "2026-10-26T11:16:08.000Z"
  }
}
```

Webhook Headers

Header	Description
`X-Sameday-Event`	Event name, e.g. `payout.pending`
`X-Sameday-Timestamp`	Unix timestamp in seconds (replay window)
`X-Sameday-Delivery`	UUID — stable across retries (use as idempotency key)
`X-Sameday-Signature`	HMAC-SHA256 signature (see below)

Retry Mechanism

- **3 attempts** per endpoint
- Retry delays: immediate → +1 second → +2 seconds
- Background retry cron (24-hour window, max 8 attempts) for persistent failures
- Your endpoint must return **HTTP 200** to acknowledge receipt

Verifying the Signature

```
const crypto = require('crypto');

function verifyWebhook(rawBody, headers, webhookSecret) {
  const timestamp = headers['x-sameday-timestamp'];
  const received  = headers['x-sameday-signature'];
  const expected  = crypto
    .createHmac('sha256', webhookSecret)
    .update(timestamp + rawBody)
    .digest('hex');
  return crypto.timingSafeEqual(Buffer.from(received, 'hex'), Buffer.from(expected, 'hex'));
}
```

Transaction Status Values

Status	Meaning	Wallet
`SUCCESS`	Transfer completed — UTR available	Debited
`PENDING`	Bank processing — poll Check Status	Held
`FAILED`	Transfer failed — auto-refunded	Refunded

Error Codes

HTTP	Code	Meaning	Fix
401	`UNAUTHORIZED`	Invalid API key, wrong signature, expired timestamp, or IP not whitelisted	Check credentials & IP whitelist
403	`FORBIDDEN`	Settlement / Payout not enabled on your account	Contact admin
403	`WALLET_FROZEN`	Wallet is frozen	Contact admin
400	`INSUFFICIENT_BALANCE`	Not enough wallet balance	Top up wallet
400	`BAD_REQUEST`	Missing or invalid field	Check `message` field
400	`SCHEME_NOT_ASSIGNED`	No payout scheme assigned to your account	Contact admin
429	`DUPLICATE`	Same account transfer within 60-second cooldown after SUCCESS	Wait and retry
400	`ACCOUNT_NOT_REGISTERED`	Account has not been registered via `POST /accounts`	Register the account first
400	`BANK_VERIFICATION_FAILED`	eKYC Hub could not confirm the account	Check account number and IFSC; use `POST /verify` first
429	`TRANSFER_IN_PROGRESS`	Same account has a PENDING transfer within 15 minutes	Wait for existing transfer or check its status
409	`DUPLICATE`	Idempotency key already used (in-flight)	Wait for original to complete
502	`REGISTRATION_FAILED`	Knteer rejected the account details	Verify account number and IFSC
503	`SERVICE_UNAVAILABLE`	Provider temporarily down	Retry after a few minutes

TRANSFER_IN_PROGRESS Error Response

When a transfer to the same account is already `PENDING` (within 15 minutes), the API returns:

```
{
  "success": false,
```

```
  "error": {
    "code": "TRANSFER_IN_PROGRESS",
    "message": "A payout to this account is already processing. Please wait for it to
complete or check its status before retrying."
  },
  "existing_transaction_id": "4e537f55-3f68-4ad8-bb55-374c4887082e",
  "existing_partner_trans_id": "KNT17290834501234",
  "existing_amount": 49969
}
```

Use `existing_transaction_id` or `existing_partner_trans_id` to check the pending transfer's status via `GET /status`.

Timestamps

All timestamps in API responses include both UTC and IST formats:

Field	Format	Example
`created_at`	ISO-8601 UTC	`2026-10-26T11:15:00.000Z`
`created_at_ist`	IST (Asia/Kolkata)	`26/10/2026, 4:45:00 pm`
`completed_at`	ISO-8601 UTC	`2026-10-26T11:15:12.000Z`
`completed_at_ist`	IST (Asia/Kolkata)	`26/10/2026, 4:45:12 pm`

> ***Note:*** *`completed_at` / `completed_at_ist` are `null` for `PENDING` transactions.*

Integration Flow (Full Example)

```
Your App                                Same Day Solution API
  |                                          |
  |── GET /balance ──────────────────────────▶ Check wallet balance
  |◀──── { balance: 9850 } ──────────────────|
  |                                          |
  |── GET /banks?popular=true ───────────────▶ Get supported banks (optional)
  |◀──── { banks: [...] } ───────────────────|
  |                                          |
  |── POST /verify ──────────────────────────▶ eKYC Hub penny-drop (₹4.72)
  |◀──── { is_valid: true,                   |
  |        account_holder_name: "RAHUL" } ──| ← bank-verified name
  |                                          |
  |── POST /accounts                         |
  |    { registration_type: "verify" } ──────▶ eKYC Hub + Knteer add-bene + ₹4.72
  |◀──── { success: true,                    |
  |        verification_charge: 4.72 } ──────|
  |                                          |
  |── POST /transfer ────────────────────────▶ Validate → Debit wallet → Call bank
  |◀──── { status: "PENDING",                |
  |        transaction_id: "abc..." } ───────|
```

```
  |                                          |
  |   (wait 2–5 minutes)                     |
  |                                          |
  |── GET /status?transactionId=abc ────────▶ Fetch live status from bank
  |◀──── { status: "SUCCESS", utr: "123" } |
  |                                          |
  ✅ Webhook also fires: payout.success ──▶ Your webhook endpoint
```

Best Practices

✅ Verify Before Register (Recommended)

Call `POST /verify` (₹4.72 — eKYC Hub penny-drop) to confirm the account exists and get the **bank-verified holder name**. Display this to the user for confirmation. Then call `POST /accounts` with `registration_type: "register"` (₹4.72, no eKYC repeat) or `registration_type: "verify"` (₹4.72 again, eKYC re-runs for a fully verified beneficiary).

✅ Register Account Before Transfer

Always call `POST /accounts` before initiating a transfer. Calling Transfer without a pre-registered account returns `ACCOUNT_NOT_REGISTERED (HTTP 400)`. Registration is ₹4.72 once — free for all future transfers to the same account.

✅ Check `already_registered` First

Before registering, you can call `GET /accounts` to list saved accounts. If the account is already there, skip registration — no charge.

✅ Always Save `transaction_id` Immediately

Store `transaction_id` in your database **before** showing a result to the user. If you get a timeout, use Check Status with this ID — never retry Transfer.

✅ Poll — Don't Re-Initiate

For `PENDING` status, call Check Status every **2–5 minutes**. Do NOT call Transfer again with the same details. The API blocks transfers to the same account while a payout is `PENDING` (15-minute cooldown) — you'll receive a `TRANSFER_IN_PROGRESS` error with the existing transaction details.

✅ Use Idempotency Keys

On network retries, include `x-idempotency-key` to prevent double-debits.

✅ Use Webhooks

Subscribe to `payout.*` events so your system updates automatically on success/failure without polling.

✅ Compute Signatures Server-Side Only

Never generate signatures in client-side JavaScript. Always compute them on your backend.

✅ Handle `FAILED` Gracefully

When a transfer returns `FAILED`, the amount is **auto-refunded** to your wallet. You can immediately retry.

⚠️ Timestamp Validity

Keep your server time synced (NTP). Timestamp must be within **5 minutes** of server time or you'll get a 401.

Testing with Postman

1. Import `SameDaySolution-Knteer-Payout-API.postman_collection.json` into Postman
2. Open **Collection Variables** and set `api_key` and `api_secret`
3. Signatures and timestamps are **auto-computed** by pre-request scripts
4. Run in order:
   - **1. Get Balance** → confirm wallet has funds
   - **2. Get Banks** → browse supported banks (optional)
   - **3. Verify Account** → eKYC Hub penny-drop (₹4.72)
   - **4a. Register Account** → ₹4.72 charged
   - **4b. List Accounts** → confirm registration
   - **5. Transfer** → initiate payout
   - **6. Check Status** → poll if PENDING
   - **7. List Transactions** → browse recent payouts

Support

Issue	Contact
API credentials / IP whitelist	Partner support — share your server IP
Wallet balance top-up	Admin / finance team
Scheme not assigned	Admin
Stuck PENDING transaction > 30 min	Support with your `transaction_id`
Failed transfer not refunded	Support with your `transaction_id`

Support email: support@samedaysolution.in